Özellikle "yapay zeka tarafından üretilmemiş belge " vurgusuna ithafen, sıfırdan implemente ettiğim Canny algoritmasının adımlarını ve neden her birine mecbur olduğumuzu özetliyorum. (Kodlarken mecburi olarak yapay zekadan yardım aldım ama bunu düzeltmeye çalışıyorummm hemde yüzde yüzz) week4-Ödev1.png dosyasında görebileceğiniz üzere, "Benim Canny" penceremdeki sonuç, OpenCV'nin yerleşik "OpenCV Canny" fonksiyonuyla neredeyse birebir aynı kalitede çalışıyor.

İşte bu temiz sonuca ulaşmak için mecburen uyguladığım adımlar ve nedenleri:

1. **. Gürültü Bastırma (5x5 Gauss Filtrelemesi)** Türev işlemleri gürültüye çok hassas olduğu için ilk olarak 5x5 boyutunda bir Gauss çekirdeği oluşturup görüntüyü konvolüsyona sokarak piksel dalgalanmalarını siliyoruz.
2. **2. Sobel Filtresi ile Gradyan Hesaplama** Kenarların konumunu ve yönünü bulmak için yatay (Kx) ve dikey (Ky) Sobel matrislerini kullanarak her pikselin gradyan büyüklüğünü (magnitude) ve açısını (direction) hesaplıyoruz.
3. **3. Maksimum Olmayanları Bastırma (Non-Maximum Suppression)** Sobel'den gelen kalın ve bulanık kenarları tek piksel kalınlığına indirmek için pikselleri hesaplanan gradyan yönlerine (0, 45, 90, 135 derece) göre komşularıyla kıyaslıyoruz. Sadece yerel maksimum değerine sahip olan tepe noktalarını tutup diğerlerini sıfırlıyoruz.
4. **4. Histerezis Eşikleme (Hysteresis Thresholding)** Belirlediğimiz alt (lowT) ve üst (highT) eşik değerleriyle çifte eşikleme yapıyoruz. Yığın (stack) yapısı kullanarak, yüksek eşiği geçen güçlü kenarları başlangıç noktası alıp, bunlara fiziksel olarak temas eden düşük eşikli zayıf pikselleri takip ederek kenara dahil ediyoruz ve kopuk/bağımsız olanları tamamen eliyoruz.

Sonuç olarak, kendi yazdığım algoritma OpenCV'nin optimizasyonlu hali ile yan yana konduğunda görsel olarak aynı incelikte, temiz ve bütünleşik bir kenar haritası vermeyi başardı.